

Working on Open Source Projects

Comprehensive Course Notes

May 2026

Modules	5 Modules
Topics	Maintainers, Licensing, GitHub Management, Building & Growing Community
Focus	Practical GitHub-based open source workflows

Module 1: The Role of Open Source Maintainers	4
1.1 Who Is a Maintainer?	4
1.2 Types of Maintainers	4
1.3 The Maintainer Mindset	4
1.4 Practical GitHub Skills for Maintainers	5
1.5 Handling Difficult Situations	5
Module 2: Choosing an Open Source License	6
2.1 Why Licensing Matters	6
2.2 The License Spectrum	6
2.3 Detailed License Breakdown	6
MIT License	6
Apache License 2.0.....	6
GNU GPL v3	7
License Decision Guide	7
2.4 Adding a License on GitHub	7
Module 3: Managing a Project on GitHub.....	8
3.1 Repository Structure Best Practices	8
3.2 Issues: Your Project's Inbox	8
3.3 Pull Requests: Code Review Workflow.....	8
3.4 GitHub Actions: Automation.....	9
3.5 Releases and Versioning	9
Module 4: Building a Community	11
4.1 Defining Your Community Identity	11
4.2 Onboarding New Contributors	11
4.3 Code of Conduct.....	12
4.4 Communication Channels.....	12
4.5 Recognition and Rewards.....	13
Module 5: Growing a Community	14
5.1 Measuring Community Health.....	14
5.2 Attracting New Contributors	14
5.3 Retaining Contributors	15
5.4 Governance Models.....	15
5.5 Sustainability and Funding.....	16
5.6 Long-Term Health Checklist	16

Quick Reference Summary	18
-------------------------------	----

Module 1: The Role of Open Source Maintainers

An open source maintainer is the backbone of any successful project. Maintainers are responsible for steering the project direction, reviewing contributions, resolving conflicts, and ensuring the long-term health of the codebase and community.

1.1 Who Is a Maintainer?

A maintainer is someone with write access to a repository who takes ongoing responsibility for its quality, direction, and community. Maintainers are not always the original author — they may be trusted contributors promoted over time.

Core Responsibilities:

- Reviewing and merging (or rejecting) pull requests
- Triaging issues: labelling, reproducing bugs, closing duplicates
- Releasing new versions and maintaining changelogs
- Communicating the project roadmap and vision
- Enforcing the Code of Conduct
- Onboarding new contributors
- Managing repository settings, branch protections, and access

1.2 Types of Maintainers

Sole Maintainer	One person makes all decisions. Common in small or personal projects. Risk: bus factor of 1.
Core Team	A small group (2–8 people) shares maintainership. Decisions made by consensus or voting.
Benevolent Dictator	One person has final say but welcomes broad input (e.g., Linus Torvalds on Linux).
Foundation Governed	A non-profit foundation (e.g., Apache, CNCF) steers the project with elected committees.

1.3 The Maintainer Mindset

The most effective maintainers balance technical rigour with empathy. They recognise that contributors are volunteering their time and approach every interaction as an opportunity to mentor and grow the community.

- Say no kindly: Most feature requests should be declined. A focused project is a healthy project.

- Write everything down: Decisions made verbally disappear. Capture rationale in issues, PRs, and ADRs.
- Lower the bus factor: Document processes and actively onboard co-maintainers.
- Sustainable pace: Burnout is the leading cause of project abandonment. Set and communicate boundaries.
- Public communication: Keep discussions open (GitHub issues, discussions) rather than in private channels.

1.4 Practical GitHub Skills for Maintainers

Branch protection rules

Navigate to Settings → Branches → Add rule. Recommended settings:

- Require pull request reviews before merging (at least 1 approval)
- Require status checks to pass before merging (CI tests)
- Require branches to be up to date before merging
- Include administrators in these restrictions

CODEOWNERS file

Place a CODEOWNERS file in root, .github/, or docs/ to automatically assign reviewers based on file paths. Example:

```
# .github/CODEOWNERS *.js @frontend-team src/api/ @backend-lead
@security-team docs/ @docs-contributors
```

GitHub Actions for maintainers

- stale bot: Automatically close issues/PRs with no activity after N days
- auto-label: Label PRs based on changed file paths
- welcome bot: Greet first-time contributors
- release-drafter: Auto-generate release notes from merged PR labels

1.5 Handling Difficult Situations

Toxic contributions

If a contributor violates the Code of Conduct: cite the specific violation, issue a warning, and escalate to a block if needed. Document all actions using GitHub's moderation tools.

Scope creep

Politely close issues and PRs outside the project's stated scope. Reference your CONTRIBUTING.md and roadmap. A well-written rejection saves more time than accepting a poor-fit contribution.

Module 2: Choosing an Open Source License

A license is a legal instrument that grants (or restricts) others' rights to use, modify, and distribute your software. Publishing code without a license means it is All Rights Reserved by default — no one can legally use it.

2.1 Why Licensing Matters

- Without a license, your code is legally unusable by others even if publicly visible
- License choice affects whether companies can build products on your project
- Incompatible licenses can prevent combining two open source projects
- Your license signals your philosophy to contributors and users

2.2 The License Spectrum

Category	Examples	Commercial Use?	Share Changes?
Permissive	MIT, Apache 2.0, BSD	Yes	No
Weak Copyleft	LGPL, MPL 2.0	Yes (with conditions)	Only the library
Strong Copyleft	GPL v2, GPL v3, AGPL	Yes, source required	Yes, entire work
Public Domain	Unlicense, CC0	Yes	No

2.3 Detailed License Breakdown

MIT License

The most popular open source license. Extremely permissive: anyone can use, copy, modify, and distribute the code, even in closed-source products. The only requirement is to retain the copyright notice.

- Best for: libraries, tools, developer utilities where maximum adoption is the goal
- Used by: React, Vue.js, Ruby on Rails, .NET Core
- Risk: Companies can build proprietary products without contributing back

Apache License 2.0

Similar to MIT but adds an explicit patent grant, protecting users from patent litigation by contributors. Important in enterprise environments.

- Best for: enterprise-friendly projects, especially those with patent-sensitive technology

- Used by: Android, Kubernetes, Apache projects
- Key advantage over MIT: explicit patent protection clause

GNU GPL v3

A strong copyleft license. Any derivative work distributed to others must also be released under GPL v3 with full source. This ensures improvements remain open.

- Best for: projects where you want to prevent proprietary use
- Used by: Linux kernel (v2), WordPress, GIMP
- AGPL variant: extends copyleft to network/SaaS use (closes the ASP loophole)

License Decision Guide

Maximum adoption including proprietary products? Use MIT or Apache 2.0

Patent protection important? Use Apache 2.0 over MIT

Keep all derivatives open source? Use GPL v3

Library linked into larger programs? Use LGPL

Project offered as a SaaS service? Use AGPL

2.4 Adding a License on GitHub

1. Go to your repository root and click 'Add file' → 'Create new file'
2. Name the file LICENSE or LICENSE.md
3. Click 'Choose a license template' to select from standard options
4. The license auto-populates with the current year and your GitHub username
5. Reference the license in your README: e.g., 'Licensed under the MIT License'

Important: Adding a license after accepting contributions requires consent from all contributors to relicense. Always choose a license before accepting your first PR.

Module 3: Managing a Project on GitHub

GitHub provides a comprehensive set of tools for managing the full lifecycle of an open source project — from tracking work and reviewing code to running automated checks and publishing releases.

3.1 Repository Structure Best Practices

README.md	Project description, installation, usage, screenshots, badges, and contribution links
LICENSE	Full license text (not just a reference)
CONTRIBUTING.md	Dev environment setup, coding standards, PR process, review timeline
CODE_OF_CONDUCT.md	Expected behaviour and enforcement procedure
CHANGELOG.md	Human-readable log of changes per release (follow Keep a Changelog format)
SECURITY.md	How to report security vulnerabilities privately
.github/	Issue templates, PR templates, CODEOWNERS, Actions workflows

3.2 Issues: Your Project's Inbox

Labels taxonomy

- Type: bug, feature, question, documentation, chore
- Priority: priority:critical, priority:high, priority:low
- Status: status:needs-triage, status:in-progress, status:blocked
- good first issue: entry-level tasks surfaced to new contributors by GitHub
- help wanted: signals you welcome outside contributions

Issue templates

Create multiple templates in `.github/ISSUE_TEMPLATE/`. Common templates: Bug Report, Feature Request, Documentation Fix, and a Security Issue template that directs reporters to a private channel.

3.3 Pull Requests: Code Review Workflow

Branch naming convention

- feature/add-dark-mode
- fix/login-redirect-loop

- docs/update-installation-guide
- chore/upgrade-dependencies

PR template checklist

6. Describe what this PR does and why
7. Link the related issue(s): 'Closes #123'
8. List any breaking changes
9. Include screenshots / screen recordings for UI changes
10. Confirm tests pass locally
11. Confirm documentation is updated

Review best practices

- Review the intent before the implementation
- Use 'Request changes' for blocking feedback, 'Comment' for suggestions
- Use GitHub's Suggestion feature in diff view for specific code improvements
- Acknowledge what is done well, not only what needs fixing
- Set a review SLA (e.g., within 7 days) and publish it in CONTRIBUTING.md

3.4 GitHub Actions: Automation

CI / Continuous Integration workflow

```
name: CI on: [push, pull_request] jobs:  test:    runs-on: ubuntu-latest
steps:  - uses: actions/checkout@v4    - uses: actions/setup-node@v4
with: { node-version: '20' }          - run: npm ci    - run: npm test
```

Dependabot for dependency updates

```
# .github/dependabot.yml version: 2 updates:  - package-ecosystem: npm
directory: "/"    schedule:    interval: weekly
```

Other useful Actions

- stale bot: Automatically close inactive issues/PRs
- auto-label: Label PRs based on changed file paths
- welcome bot: Greet first-time contributors
- release-drafter: Auto-generate release notes from PR labels

3.5 Releases and Versioning

Semantic Versioning (SemVer): MAJOR.MINOR.PATCH

- MAJOR: breaking changes (e.g., 1.x.x → 2.0.0)
- MINOR: new backward-compatible features (e.g., 1.3.0 → 1.4.0)
- PATCH: backward-compatible bug fixes (e.g., 1.4.2 → 1.4.3)
- Pre-release suffixes: 1.5.0-alpha.1, 1.5.0-beta.2, 1.5.0-rc.1

Creating a GitHub Release

12. Tag the commit: `git tag v1.4.0 && git push --tags`
13. Go to Releases → Draft a new release

14. Select the tag, add a title and release notes
15. Attach compiled binaries or packages as assets if applicable
16. Mark as pre-release if not production-ready

Module 4: Building a Community

Code without a community is just a repository. The longevity and success of an open source project depends as much on the human infrastructure around it as on the technical quality of the code.

4.1 Defining Your Community Identity

Write a vision statement

One or two sentences that answer: What problem does this solve? Who is it for? What makes it different? Example: 'A zero-config JavaScript bundler for the 99% of apps that don't need a complex build system.'

Define your audience

End users	People who use your project's output (e.g., website visitors benefiting from the library)
Developer users	Developers who depend on your package in their own projects
Contributors	Developers who submit code, docs, or design to the project
Advocates	Community members who write blog posts, give talks, help newcomers in forums

4.2 Onboarding New Contributors

The contributor funnel

17. User discovers the project (search, blog post, talk, dependency)
18. User reads README and stars the repository
19. User encounters a bug or wants a feature
20. User reads CONTRIBUTING.md
21. User sets up the dev environment
22. User finds a 'good first issue'
23. User opens a PR
24. Maintainer reviews quickly (within a week)
25. PR is merged — contributor feels valued
26. Contributor becomes a regular

Good first issues

Maintain a backlog of at least 5 open 'good first issue' items at all times. Ideal candidates are small, self-contained, well-explained tasks with clear acceptance criteria and hints toward the relevant code files.

CONTRIBUTING.md must answer:

- How do I set up the development environment step by step?
- How do I run the tests?
- What is your coding style / linting setup?
- What is the PR review process and timeline?
- Are there areas where you do not want contributions?
- How do I ask questions?

4.3 Code of Conduct

A Code of Conduct makes explicit the expected behaviour in your community and the consequences for violations. Without one, you are implicitly signalling that harassment will be tolerated.

Recommended: Contributor Covenant (contributorcovenant.org)

- Add an enforcement contact email (a private address maintainers monitor)
- Specify consequences: warning, temporary ban, permanent ban
- Respond to reports within 48 hours
- Document your process even if you never need to use it

GitHub enforcement tools

- Block user: Prevents a user from opening issues, PRs, or commenting
- Limit interactions: Temporarily restrict interactions to existing contributors
- Lock conversation: Prevents further comments on a specific thread
- Hide comment: Collapses a comment behind a 'hidden' label

4.4 Communication Channels

GitHub Discussions	Best for: Q&A, ideas, announcements. Keeps discussion linked to code. Recommended as the primary channel.
Discord	Best for: real-time chat, support, informal community. Good for active projects with high daily traffic.
Slack	Enterprise-friendly. Good if your users are corporate developers. Paid for full message history.
Mailing list	Traditional. Preferred by privacy-conscious or email-centric communities (Apache, Linux projects).
Forum (Discourse)	Long-form threaded discussions. Good for complex troubleshooting and knowledge base building.

Golden rule: fewer channels, better moderated. Start with GitHub Issues + GitHub Discussions. Add chat only when there is clear demand.

4.5 Recognition and Rewards

- All Contributors bot: Recognises all contribution types (code, docs, design, translation) in README
- Shout-outs in release notes: Mention contributor usernames in each release
- Write-ups in project blog: Feature substantial contributors with their story
- Maintainer track: Create a clear path from contributor to co-maintainer for trusted community members

Module 5: Growing a Community

Building a community is about creating the foundation. Growing it is about sustained, intentional effort to reach new people, retain existing contributors, ensure project health, and build towards long-term sustainability.

5.1 Measuring Community Health

Contributors	Total unique contributors over time; new vs returning ratio
Commits	Commit frequency; distribution across contributors (bus factor indicator)
Issues	Open vs closed ratio; average time to first response; average time to close
Pull Requests	Median time from opening to merge; PR acceptance rate
Traffic	Unique visitors, clone count, referring sites (reveals discovery channels)
Stars & Forks	Growth trend indicator and viral event tracker

The CHAOSS (Community Health Analytics in Open Source Software) project provides a standardised set of metrics and open source tools (Augur, GrimoireLab) for measuring project health at scale.

5.2 Attracting New Contributors

Discoverability on GitHub

- Add descriptive topics/tags to your repository (Settings → Topics)
- Write a detailed repository description (shows in GitHub search results)
- Keep your README polished — it is your landing page
- Pin the repository to your GitHub profile or organisation page

Content marketing

- Blog post announcing each major release with real use cases
- Tutorial series: 'Build X with [your-project]'
- Video walkthrough on YouTube for visual learners
- Dev.to, Hashnode, or Medium cross-posts to reach developer audiences
- Twitter/X threads breaking down interesting technical decisions

Events and hackathons

- Submit CFPs to relevant conferences: KubeCon, PyCon, JSConf, RailsConf, etc.
- Participate in Hacktoberfest: massive first-time contributor influx every October
- Sponsor or co-run a hackathon around your tool

5.3 Retaining Contributors

Response time is everything

A first-time contributor who waits more than two weeks for review feedback is unlikely to contribute again. Set a stated review SLA (e.g., ‘we aim to respond within 7 days’) and measure it.

The contributor journey

Lurker	Reads issues and PRs, uses the project. Nurture: excellent documentation, welcoming tone.
Reporter	Opens issues. Nurture: fast, respectful responses; thank them for the report.
First-time contributor	Opens first PR. Nurture: quick review, clear feedback, celebrate the merge.
Regular contributor	Multiple merged PRs. Nurture: give ownership over specific areas; invite to triage.
Maintainer	Trusted co-maintainer with write access. Nurture: include in decisions, credit publicly.

Avoiding contributor burnout

- Rotate the burden of issue triage across multiple maintainers
- Declare a ‘maintenance mode’ or ‘hiatus’ publicly rather than going dark
- Use GitHub Discussions for community support — let power users answer questions
- Set clear scope boundaries so you are not overwhelmed by out-of-scope requests

5.4 Governance Models

BDFL (Benevolent Dictator for Life)

Single person makes final calls. Simple and decisive. Fragile if the BDFL burns out or leaves. Examples: Python (originally Guido van Rossum), Django.

Meritocracy / Contributor Ladder

Contributors earn greater responsibility and access as they demonstrate consistent good work. Apache Software Foundation pioneered this model. Roles: Contributor → Committer → PMC Member.

Steering Committee

Elected or appointed committee makes binding decisions via consensus or vote. Used by Node.js (TSC), Kubernetes (Steering Committee), Rust (Core Team).

Document your governance in GOVERNANCE.md:

- Who are the current maintainers?
- How are decisions made (consensus, vote, BDFL)?
- How are new maintainers added?
- How are disputes resolved?
- How is the project funded?

5.5 Sustainability and Funding

GitHub Sponsors

Enable GitHub Sponsors to accept recurring funding from individuals and companies. GitHub takes no fee. Add FUNDING.yml to display a 'Sponsor' button on your repository.

Other funding avenues

- Open Collective: Transparent fund management for communities
- Tidelift: Pays maintainers for security and maintenance commitments in enterprise subscriptions
- Foundation membership: Donate the project to Linux Foundation, Apache, CNCF for governance and infrastructure
- Dual licensing: GPL for free, commercial license for proprietary use (MySQL model)
- Open core: Free core features, paid premium features (GitLab model)
- Consulting and support contracts: Offer paid support (Red Hat model)

5.6 Long-Term Health Checklist

☐	README is up to date and accurately describes the project
☐	CONTRIBUTING.md has clear step-by-step setup instructions
☐	CI passes on the main branch at all times
☐	Dependencies are up to date (Dependabot enabled)
☐	At least 3 labelled 'good first issue' items are open
☐	Average time to first response on issues is under 14 days
☐	A Code of Conduct is in place with active enforcement contacts
☐	More than one person has maintainer/commit access
☐	A CHANGELOG.md is maintained and linked from releases
☐	Governance is documented in GOVERNANCE.md
☐	A SECURITY.md file explains how to report vulnerabilities privately

☐	The project has a public roadmap or milestone board
☐	A funding or sustainability plan is in place or actively being sought

Quick Reference Summary

Module 1	Maintainers own project direction, quality, and community. Use CODEOWNERS, branch protection, issue templates. Distribute responsibility and document everything to guard against burnout.
Module 2	Always add a license before accepting contributions. MIT/Apache 2.0 for permissive; GPL for copyleft; AGPL for SaaS copyleft. Add via GitHub's license picker.
Module 3	Structure repo with README, CONTRIBUTING, CoC, CHANGELOG, SECURITY. Use branch protection + CI via GitHub Actions. Release with SemVer and create GitHub Releases.
Module 4	Onboard via 'good first issues', welcoming CONTRIBUTING.md, and fast PR reviews. Enforce a Code of Conduct. Recognise all contribution types.
Module 5	Track health metrics. Grow via content, conferences, and discoverability. Retain by responding fast and giving ownership. Formalise governance. Fund via Sponsors, foundations, or dual licensing.

Key GitHub Resources

- choosealicense.com — Interactive license picker by GitHub
- contributorcovenant.org — Standard Code of Conduct
- keepachangelog.com — CHANGELOG format guide
- semver.org — Semantic versioning specification
- allcontributors.org — Recognise all contributor types
- chaoss.community — Open source health metrics framework
- opensource.guide — Comprehensive guides by GitHub (highly recommended)
- github.com/sponsors — Set up funding for your project